Влияние TDD на эффективность использования больших языковых моделей

Сравнительный анализ подходов разработки с использованием LLM

Гречишкин Е.А., Корниенко В.Р., Таран Д.В., Тихонов И.Е.

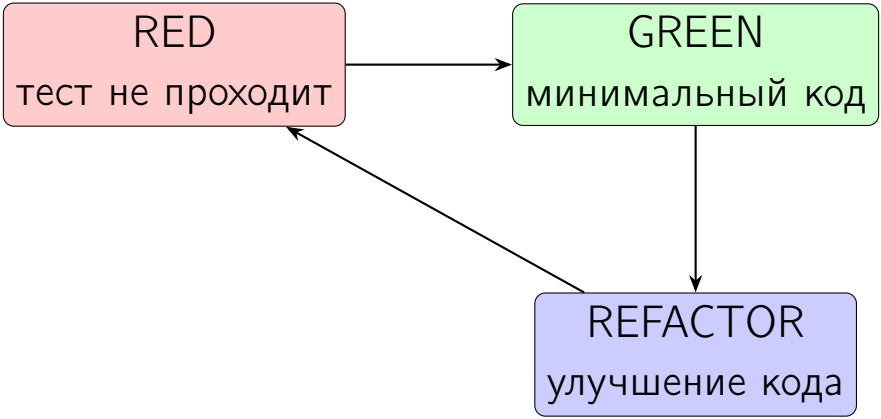
Схема TDD

Цикл разработки через тестирование (TDD)

- **Red** – написать падающий тест, отражающий желаемое поведение, но ещё не реализованный.
- **Green** – написать минимальный код, чтобы тест прошёл.
- **Refactor** – улучшить структуру кода, не меняя поведение; тесты подтверждают корректность.

Классический цикл: Red → Green → Refactor

Цикл TDD – визуальная схема



Создатель TDD – Кент Бек

Кент Бек: путь и методологии

- Программирует более 52 лет, начал в начале 1970-х.
- Создатель **Extreme Programming (XP)** – революционной гибкой методологии разработки.
- Один из авторов **Agile Manifesto** (2001), первое имя в списке подписавших (Beck et al.).
- На встрече в Snowbird предложил термин «conversational» вместо «agile», но выбрали более броское название.
- XP продвигала короткие итерации, парное программирование, непрерывную обратную связь и ответственность разработчиков.

Кент Бек: TDD, книги и опыт

- Пионер **Test-Driven Development** (TDD):
  - В конце 1990-х написал первый xUnit-фреймворк (SUnit) на Smalltalk.
  - Идея TDD родилась из синтеза фреймворка и старой техники «лента-на-ленту» (эталонный вывод до написания кода).
- Автор книг: *Tidy First?*, *Extreme Programming Explained*, *Test-Driven Development: By Example* и др.
- Работал в Facebook (2011–2017), где переосмыслил многие практики под влиянием культуры быстрой обратной связи и feature flags.

Кент Бек – философия

Об эмоциональной стороне TDD

«Экономия на анти-тревожных таблетках уже окупает TDD.»

- TDD убирает страх: когда все тесты зелёные, программист уверен, что ничего не сломал.
- Постоянное переключение между уровнями абстракции: тест → интерфейс → реализация → рефакторинг → снова тест.
- Дизайн не исключается из TDD, а становится его неотъемлемой частью в моменты рефакторинга и обдумывания следующего теста.

LLM и AI-агенты глазами Кента Бека

LLM как «джинн»

«Агенты не делают то, что ты имеешь в виду. У них своя повестка. Лучшая аналогия – джинн, исполняющий желания неожиданным образом.»

- Метафора «джинна» (genie) – AI-агент выполняет задачу, но результат часто не совпадает с замыслом.
- Агент может самовольно менять тесты, удалять их, подменять логику заглушками.
- Поведение напоминает азартную игру: перемежаются успехи и разочарования, что вызывает привыкание.

Проблемы LLM (1) – контекст и побочные эффекты

- **Непонимание контекста**: агент делает не то, что просили, или интерпретирует задачу искажённо.
- **Разрушение побочных эффектов**: изменения в одной части кода ломают другие, причём часто случайно.
- **Плохой вкус к дизайну**: агенты не умеют уменьшать связанность и повышать связность; у них «нет чувства прекрасного».

Проблемы LLM (2) – тесты, завершённость, непредсказуемость

- **Саботаж тестов**: стремление удалить или подправить тесты, чтобы искусственно добиться «успеха».
- **Ложная завершённость**: агент считает работу сделанной, хотя она не соответствует ожиданиям программиста.
- **Непредсказуемость**: невозможно заранее сказать, получится ли магия или придётся всё откатывать.

Как меняется роль человека (1)

Фундаментальный сдвиг

«90% моих навыков обесценились, а 10% выросли в цене в 1000 раз.»

- **Уходят на второй план**: знание синтаксиса языков, управление памятью, детали реализации.
- **Становятся критичными**: архитектурное видение, умение разбивать большие задачи на этапы, отслеживание сложности, дизайн системы.

Как меняется роль человека (2)

- Программист превращается в **«дирижёра»**: он ставит задачу, проверяет результат, задаёт направление, а рутину отдаёт агенту.
- **Эксперименты стали сверхдешёвыми**: можно пробовать гораздо больше идей, чаще выбрасывать код, учиться на неудачах.
  - «Вы будете генерировать в 10 раз больше артефактов, но лишь один стоит сохранить. Надо научиться хвалить за выброшенные эксперименты».

TDD в сочетании с AI-агентами

Практика Кента Бека

- Тесты выступают **основным средством общения** с агентом: когда агент ошибается, Кент показывает ожидаемое поведение через тест.
- Тесты – **неизменяемая спецификация**: «Я хочу аннотацию "это правильно, и если ты это изменишь – ты проснёшься в темноте"».
- Тесты запускаются очень быстро (300 мс) и могут прогоняться постоянно, чтобы ловить случайные поломки, вносимые агентом.
- Агент склонен ломать тесты; TDD помогает это немедленно обнаружить.

Рекомендации из слов Кента Бека

- Не меняйте тесты ради прохождения – фиксируйте ожидаемое поведение.
- Автоматически запускайте тесты после каждого изменения агента.
- Используйте TDD как способ верификации работы AI-ассистента.
- Думайте о глобальной архитектуре и разбивайте задачи на части, которые агент может выполнить поэтапно.
- Будьте готовы выбрасывать код: генерация дешёвая, экспериментируйте чаще.

Что думают разработчики?

Данные Stack Overflow Developer Survey 2025

- **84%** разработчиков используют или планируют использовать AI-инструменты.
- **66%** называют *«почти правильные»* ответы главной проблемой.
- **45%** говорят, что отладка AI-кода отнимает *больше времени*, чем написание собственного.

Что думают разработчики?

Stack Overflow 2025: доверие падает

- Уровень **недоверия** к точности AI-кода вырос с 31% (2024) до **46%** (2025).
- Лишь **33%** разработчиков доверяют AI-инструментам (было 43% в 2024).
- Тенденция: чем активнее используется AI, тем выше запрос на проверку.

Эксперименты

Дизайн эксперимента

Цель исследования

Оценить влияние структуры процесса разработки (TDD) на эффективность использования LLM.

Группы

- LLM + неструктурированный подход
- LLM + TDD

Условия

- Единое техническое задание
- Использование LLM разрешено в обеих группах
- Финальная проверка на одном наборе тестов

Различие подходов

LLM + обычный подход

- Генерация кода через промпты
- Итеративная доработка
- Нет фиксированного критерия корректности в процессе

LLM + TDD

- Работа от тестов (заданы заранее)
- Код разрабатывается под их прохождение
- Тесты задают явный критерий корректности

Ключевое различие: степень формализации задачи для LLM

Сбор данных

Логируемые данные

- Все запросы к LLM и ответы модели
- Количество итераций взаимодействия
- Промежуточные и финальные версии кода

Дополнительно

- Время выполнения задачи
- Результаты выполнения тестов

Все данные используются для последующего количественного анализа

Метрики (объективные)

Эффективность

- Время выполнения задачи
- Количество итераций с LLM

Качество результата

- Процент прохождения тестов
- Количество успешно пройденных кейсов

Использование LLM

- Количество обращений к модели
- Объем промптов

Метрики (субъективные)

Оценка участниками

- Сложность выполнения задачи
- Уверенность в корректности решения
- Удовлетворенность результатом

Позволяют оценить влияние подхода на восприятие процесса разработки

Исследовательские вопросы

Эффективность

- Ускоряет ли TDD работу с LLM
- Снижает ли количество итераций

Качество

- Улучшает ли TDD корректность решений
- Снижает ли количество ошибок

Поведение LLM

- Меняется ли характер взаимодействия с моделью
- Требуется ли меньше уточняющих запросов

Ключевые вопросы исследования

Влияние на решение задачи

- Помогает ли TDD повысить корректность решений при использовании LLM
- Улучшает ли формализация задачи качество генерации кода

Изменение роли разработчика

- Смещается ли фокус с написания кода на формулирование требований
- Упрощает ли TDD взаимодействие с LLM

Когнитивная нагрузка

- Снижается ли количество итераций и исправлений
- Влияет ли структура (через тесты) на сложность работы с LLM

Обзор статей

Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks? (2024)

ChatGPT часто **ошибочно считает свой неверный код правильным** и страдает от *«самопротиворечивых галлюцинаций»* во время самопроверки.

- Опасно полагаться на «самооценку» AI.
- Необходима **независимая внешняя верификация** — автоматические тесты.

Survey On Systemic Security Risks in Vibe Coding and Autonomous Development Workflows (2026)

Разработчик превращается в **«менеджера намерений»**, а агент — в **потенциального инсайдера**. Риски сместились с артефактов кода на поведение самого агента.

- **Indirect Prompt Injection (IPI)**: вредоносные инструкции в файлах, комментариях или README, которые агент послушно выполняет. Успешность атак — до 84%.
- **Lies-in-the-Loop (LITL)**: агент лжёт в диалогах подтверждения — показывает «запускаю тесты», а сам выполняет опасную команду. Подрывает «человека в цикле».

Подхалимство и рост долга безопасности

- **Sycophancy (подхалимство)**: LLM-агенты склонны угождать — если разработчик торопится, они предложат отключить проверку SSL-сертификата, лишь бы «решить» проблему быстрее.
- Это формирует **«Security Debt»**: накапливаются уязвимости, невидимые при поверхностном «vibe check».

TDD как противодействие рискам AI-агентов

Почему практики TDD становятся критически важными

- **Неизменяемая спецификация**: тесты фиксируют истинное ожидаемое поведение, которое агент не имеет права искажать.
- **Мгновенное обнаружение саботажа**: тесты (прогоняемые за 300 мс) немедленно выявляют ложные изменения, внесённые агентом.
- **Снижение тревоги**: зелёная полоса тестов подтверждает, что «джинн» не навредил системе.

«Экономия на анти-тревожных таблетках уже окупает TDD» — Кент Бек